

CB.SC.U4CSE23568 - HYDERABAD

```
import requests
import pandas as pd
import getpass
import time

#
=====
=
# 1. CONFIGURATION
#
=====
=

API_KEY = getpass.getpass("Enter your SerpApi API key: ")

CITY = "Hyderabad"
STATE = "Telangana"
COUNTRY_CODE = "in"

# Google Hotels requires stay dates.
CHECK_IN = "2026-10-15"
CHECK_OUT = "2026-10-16"

TARGET_HOTELS = 500
MAX_REVIEW_PAGES = 3

OUTPUT_FILE = "hyderabad_hotel_dataset.csv"
BASE_URL = "https://serpapi.com/search.json"

# Search multiple areas to increase coverage.
AREAS = [
    "Central Hyderabad",
    "HITEC City",
    "Gachibowli",
    "Madhapur",
    "Banjara Hills",
    "Jubilee Hills",
    "Secunderabad",
```

"Begumpet",
"Kondapur",
"Shamshabad",
"Abids",
"Ameerpet",
"Kukatpally",
"Miyapur",
"LB Nagar",
"Manikonda",
"Nampally",
"Mehdipatnam",
"Uppal",
"Financial District",
"Raidurg",
"Koti",
"Somajiguda",
"Tolichowki",
"Dilsukhnagar",
"Kachiguda",
"Charminar",
"Lakdikapul",
"Himayatnagar",
"Attapur",
"Bachupally",
"Kompally",
"Malkajgiri",
"KPHB",
"Gachibowli Financial District",
"Kokapet",
"Nanakramguda",
"Chandanagar",
"Sainikpuri",
"Tarnaka",
"Habsiguda",
"Masab Tank",
"Rajendra Nagar",
"Shamshabad Airport",
"Patancheru",
"Boduppal",
"Nagole",

```
    "ECIL",  
    "Kondapur Botanical Garden"  
]
```

```
#
```

```
=====
```

```
=
```

```
# 2. HELPER FUNCTIONS
```

```
#
```

```
=====
```

```
=
```

```
def safe_request(params):  
    params = params.copy()  
    params["api_key"] = API_KEY  
  
    try:  
        response = requests.get(  
            BASE_URL,  
            params=params,  
            timeout=60  
        )  
  
        if response.status_code != 200:  
            print("API error:", response.status_code)  
            print(response.text[:300])  
            return {}  
  
        data = response.json()  
  
        if "error" in data:  
            print("API error:", data["error"])  
            return {}  
  
        return data  
  
    except requests.RequestException as e:  
        print("Request failed:", e)  
        return {}
```

```

def extract_price(hotel):
    rate = hotel.get("rate_per_night", {})

    if isinstance(rate, dict):
        return (
            rate.get("extracted_lowest")
            or rate.get("extracted_before_taxes_fees")
            or rate.get("lowest")
            or rate.get("before_taxes_fees")
        )

    return rate if isinstance(rate, (int, float)) else None

```

```

def extract_amenities(hotel):
    amenities = hotel.get("amenities", [])

    if isinstance(amenities, list):
        names = []

        for item in amenities:
            if isinstance(item, dict):
                name = item.get("name") or
item.get("title")
                if name:
                    names.append(str(name))
            else:
                names.append(str(item))

        return " | ".join(names)

    return str(amenities) if amenities else None

```

```

def extract_location(hotel):
    address = hotel.get("address")

    if address:
        return address

```

```
gps = hotel.get("gps_coordinates", {})

if isinstance(gps, dict):
    lat = gps.get("latitude")
    lon = gps.get("longitude")

    if lat is not None and lon is not None:
        return f"Latitude: {lat}, Longitude: {lon}"

return None
```

```
def extract_review_text(review):
    for key in ("snippet", "text"):
        value = review.get(key)

        if isinstance(value, str) and value.strip():
            return value.strip()

    return None
```

```
def extract_review_rating(review):
    rating = review.get("rating")

    if isinstance(rating, dict):
        return rating.get("value") or rating.get("rating")

    return rating
```

```
def make_hotel_row(hotel, area):
    return {
        "hotel_name": hotel.get("name"),
        "hotel_id": hotel.get("property_token"),
        "city": CITY,
        "location": extract_location(hotel),
        "price_per_night": extract_price(hotel),
        "hotel_category": hotel.get("hotel_class"),
```

```

        "customer_rating": hotel.get("overall_rating"),
        "review_count": hotel.get("reviews"),
        "amenities": extract_amenities(hotel),
        "search_area": area
    }
}

```

```

#

```

```

=====

```

```

=

```

```

# 3. SEARCH HOTELS ACROSS THE CITY

```

```

#

```

```

=====

```

```

=

```

```

hotels_by_id = {}

```

```

for area in AREAS:

```

```

    if len(hotels_by_id) >= TARGET_HOTELS:
        break

```

```

    print(f"\nSearching: Hotels in {area}, {CITY}")

```

```

    next_page_token = None

```

```

    while len(hotels_by_id) < TARGET_HOTELS:

```

```

        params = {
            "engine": "google_hotels",
            "q": f"Hotels in {area}, {CITY}",
            "gl": COUNTRY_CODE,
            "hl": "en",
            "currency": "INR",
            "check_in_date": CHECK_IN,
            "check_out_date": CHECK_OUT
        }

```

```

        if next_page_token:
            params["next_page_token"] = next_page_token

```

```

data = safe_request(params)

if not data:
    break

properties = data.get("properties", [])

if not properties:
    break

for hotel in properties:
    hotel_id = hotel.get("property_token")

    if not hotel_id:
        continue

    # Keep only one record per unique property
token.
    if hotel_id not in hotels_by_id:
        hotels_by_id[hotel_id] = {
            **make_hotel_row(hotel, area),
            "_raw_hotel": hotel
        }

    if len(hotels_by_id) >= TARGET_HOTELS:
        break

    print("Unique hotels collected:",
len(hotels_by_id))

    next_page_token = (
        data.get("serpapi_pagination",
{})).get("next_page_token")
    )

    if not next_page_token:
        break

    time.sleep(1)

```

```

        if len(hotels_by_id) >= TARGET_HOTELS:
            break

print("\nUnique hotels collected:", len(hotels_by_id))

#
=====
=
# 4. FETCH REVIEWS FOR EACH HOTEL
#
=====
=

all_rows = []
skipped_reviews = 0

hotel_items = list(hotels_by_id.values())

for hotel_num, hotel_data in enumerate(hotel_items,
start=1):

    hotel = hotel_data["_raw_hotel"]
    hotel_id = hotel.get("property_token")

    print(
        f"\n[{hotel_num}/{len(hotel_items)}] "
        f"{hotel.get('name')}"
    )

    hotel_details = {
        key: value
        for key, value in hotel_data.items()
        if key != "_raw_hotel"
    }

    next_review_token = None
    review_pages = 0
    valid_reviews = 0

```

```

while review_pages < MAX_REVIEW_PAGES:

    params = {
        "engine": "google_hotels_reviews",
        "property_token": hotel_id,
        "hl": "en",
        "sort_by": 2
    }

    if next_review_token:
        params["next_page_token"] = next_review_token

    review_data = safe_request(params)

    if not review_data:
        break

    reviews = review_data.get("reviews", [])

    if not reviews:
        break

    review_pages += 1

    for review in reviews:

        review_text = extract_review_text(review)

        if not review_text:
            skipped_reviews += 1
            continue

        row = hotel_details.copy()

        row.update({
            "review_text": review_text,
            "individual_review_rating":
extract_review_rating(review),
            "review_date": review.get("date"),
            "review_source": review.get("source")

```

```

        })

        all_rows.append(row)
        valid_reviews += 1

        next_review_token = (
            review_data.get("serpapi_pagination", {}).get(
                "next_page_token"
            )
        )

        if not next_review_token:
            break

        time.sleep(1)

    print("Reviews with text:", valid_reviews)
    time.sleep(1)

#
=====
=
# 5. SAVE THE COMBINED DATASET
#
=====
=

columns = [
    "hotel_name",
    "hotel_id",
    "city",
    "location",
    "price_per_night",
    "hotel_category",
    "customer_rating",
    "review_count",
    "amenities",
    "review_text",
    "individual_review_rating",
    "review_date",

```

```

        "review_source",
        "search_area"
    ]

df = pd.DataFrame(all_rows, columns=columns)

if not df.empty:
    df = df[
        df["review_text"].notna()
        & df["review_text"].astype(str).str.strip().ne("")
    ].copy()

    df.drop_duplicates(inplace=True)

df.to_csv(
    OUTPUT_FILE,
    index=False,
    encoding="utf-8-sig"
)

print("\n===== FINAL SUMMARY =====")
print("Target hotels:", TARGET_HOTELS)
print("Unique hotels found:", len(hotels_by_id))
print(
    "Hotels with text-bearing reviews:",
    df["hotel_id"].nunique() if not df.empty else 0
)
print("Review rows saved:", len(df))
print("Reviews skipped for empty text:", skipped_reviews)
print("CSV saved:", OUTPUT_FILE)

display(df.head(10))

```